Introduction & Modelling Approach

Automating Loan Approvals with Machine Learning

- **The Problem:** Traditional loan evaluation relies on manual assessments making it slow, expensive, prone to human error, and subject to subjective biases across demographic factors.

- **The Solution:** A scalable, data-driven machine learning framework that standardizes loan risk assessment using real-time applicant data.

- **Preprocessing Workflow:**
  - **Data Sanitization & Feature Engineering:** Cleans string formatting bugs, aggregates scattered asset types into `total_assets_value`, and builds critical risk ratios (`loan_to_income`, `loan_to_asset`).
  - **Leak-Free Pipelines:** Integrates scaling (`StandardScaler`) and encoding (`OneHotEncoder`) inside sklearn `Pipeline` objects to prevent test-set data leakage.

- **Benchmarking models:**
  - **Logistic Regression:** Serves as a transparent, fast, and fully interpretable linear baseline.
  - **Random Forest & XGBoost:** Ensembled tree models chosen to capture non-linear interactions and complex decision thresholds without assuming linear relationships in the financial features.

[4]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, roc_auc_score,
    confusion_matrix, classification_report
)

# classifiers
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
try:
    from xgboost import XGBClassifier
    has_xgboost = True
except ImportError:
    has_xgboost = False
```

Environment & Library Setup

Data & Visualization: pandas and numpy for data manipulation; matplotlib and seaborn for plotting.
Preprocessing & Pipeline: StandardScaler (feature scaling), OneHotEncoder (categorical variables), and Pipeline to prevent **data leakage**.
Classifiers: Logistic Regression (linear baseline), Random Forest & XGBoost (tree ensembles for non-linear patterns).
Evaluation: Multi-metric setup (ROC-AUC, Precision, Recall, F1-score) for comprehensive risk assessment.

[5]:
```python
# data cleaning

df = pd.read_csv("loan_approval_dataset.csv")

# clean leading/trailing spaces in column names and text entries
df.columns = df.columns.str.strip()
for col in df.select_dtypes(include='object').columns:
    df[col] = df[col].astype(str).str.strip()
```

[6]:
```python
# feature engg
# combine asset values into total wealth
df['total_assets_value'] = (
    df['residential_assets_value'] +
    df['commercial_assets_value'] +
    df['luxury_assets_value'] +
    df['bank_asset_value']
)

# compute key financial risk ratios
df['loan_to_income_ratio'] = df['loan_amount'] / (df['income_annum'] + 1e-5)
df['loan_to_asset_ratio'] = df['loan_amount'] / (df['total_assets_value'] + 1e-5)
```

Data Cleaning & Feature Engineering

Data Loading & Cleaning: Reads the CSV dataset and strips whitespace from headers and string values to prevent hidden formatting bugs.
Asset Aggregation: Sums residential, commercial, luxury, and bank assets into a single total_assets_value feature to quantify total collateral.
Financial Risk Ratios: Computes key financial metrics (loan_to_income_ratio and loan_to_asset_ratio) to evaluate applicant debt burden and risk exposure.
Zero-Division Protection: Adds a small constant (1e-5) to denominators to prevent division-by-zero errors.

[7]:
```
#features and target
X = df.drop(columns=['loan_id', 'loan_status'])
y = df['loan_status'].map({'Approved': 1, 'Rejected': 0})  # 1: Approved, 0: Rejected

# identify column types for Preprocessing Pipeline
num_features = X.select_dtypes(include=['int64', 'float64']).columns.tolist()
cat_features = X.select_dtypes(include=['object']).columns.tolist()

# Train-Test Split (Stratified to maintain class balance)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
```

[8]:
```
# preprocessing
# preprocessor scales numerical features and One-Hot Encodes categorical features
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), num_features),
        ('cat', OneHotEncoder(drop='first', handle_unknown='ignore'), cat_features)
    ]
)
```

Dataset Splitting & Preprocessing Pipeline

Feature & Target Definition: Drops non-predictive identifiers (loan_id) and maps the target variable (loan_status) to binary labels (1: Approved, 0: Rejected).
Automated Type Identification: Dynamically segregates numerical and categorical columns for tailored preprocessing.
Stratified Train-Test Split: Allocates 80% data for training and 20% for testing while preserving the original approval-to-rejection ratio (stratify=y).
ColumnTransformer Integration: Standardizes numerical variables using StandardScaler and One-Hot Encodes categorical features while dropping the first level (drop='first') to prevent multicollinearity.

[9]:
```python
#model eval and b-marking
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42)
}

if has_xgboost:
    models['XGBoost'] = XGBClassifier(eval_metric='logloss', random_state=42)

results = []

print("="*60)
print("MODEL BENCHMARK RESULTS")
print("="*60)

for name, clf in models.items():
    # Construct complete pipeline
    pipeline = Pipeline(steps=[
        ('preprocessor', preprocessor),
        ('classifier', clf)
    ])

    # Train
    pipeline.fit(X_train, y_train)

    # Predict
    y_pred = pipeline.predict(X_test)
    y_proba = pipeline.predict_proba(X_test)[:, 1]

    # Evaluate Metrics
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    auc = roc_auc_score(y_test, y_proba)

    results.append({
        'Model': name,
        'Accuracy': acc,
        'Precision': prec,
        'Recall': rec,
        'F1-Score': f1,
        'ROC-AUC': auc
    })

    print(f"\n--- {name} ---")
    print(f"Accuracy : {acc:.4f} | ROC-AUC: {auc:.4f}")
    print("\nClassification Report:\n", classification_report(y_test, y_pred, target_names=['Rejected', 'Approved']))

# Benchmark Comparison Table
results_df = pd.DataFrame(results).sort_values(by='ROC-AUC', ascending=False)
print("\n" + "="*60)
print("SUMMARY COMPARISON")
print("="*60)
print(results_df.to_string(index=False))
```

Model Training & Benchmarking Pipeline

Multi-Model Setup: Initializes three algorithms—Logistic Regression (linear), Random Forest (ensemble), and optionally XGBoost (gradient boosting)—to compare performance.

Leak-Free Training: Wraps preprocessing and classification inside Pipeline objects so data scaling/encoding fits exclusively on training folds during fit().

Probability & Class Predictions: Extracts both discrete class predictions (y_pred) and continuous approval probabilities (y_proba) for risk assessment.

Comprehensive Metrics: Computes Accuracy, Precision, Recall, F1-Score, and ROC-AUC to evaluate true positive/false positive trade-offs.

Automated Leaderboard: Compiles performance metrics into a clean summary DataFrame sorted by ROC-AUC score for instant benchmarking.

```
MODEL BENCHMARK RESULTS
============================================================

--- Logistic Regression ---
Accuracy : 0.9075 | ROC-AUC: 0.9727

Classification Report:
              precision    recall  f1-score   support

    Rejected       0.89      0.86      0.88       323
    Approved       0.92      0.94      0.93       531

    accuracy                           0.91       854
   macro avg       0.90      0.90      0.90       854
weighted avg       0.91      0.91      0.91       854


--- Random Forest ---
Accuracy : 0.9988 | ROC-AUC: 1.0000

Classification Report:
              precision    recall  f1-score   support

    Rejected       1.00      1.00      1.00       323
    Approved       1.00      1.00      1.00       531

    accuracy                           1.00       854
   macro avg       1.00      1.00      1.00       854
weighted avg       1.00      1.00      1.00       854


--- XGBoost ---
Accuracy : 0.9977 | ROC-AUC: 1.0000

Classification Report:
              precision    recall  f1-score   support

    Rejected       1.00      1.00      1.00       323
    Approved       1.00      1.00      1.00       531

    accuracy                           1.00       854
   macro avg       1.00      1.00      1.00       854
weighted avg       1.00      1.00      1.00       854


============================================================
SUMMARY COMPARISON
============================================================
               Model  Accuracy  Precision   Recall  F1-Score  ROC-AUC
       Random Forest  0.998829   0.998120 1.000000  0.999059 1.000000
             XGBoost  0.997658   0.998117 0.998117  0.998117 0.999965
Logistic Regression  0.907494   0.915441 0.937853  0.926512 0.972690
```

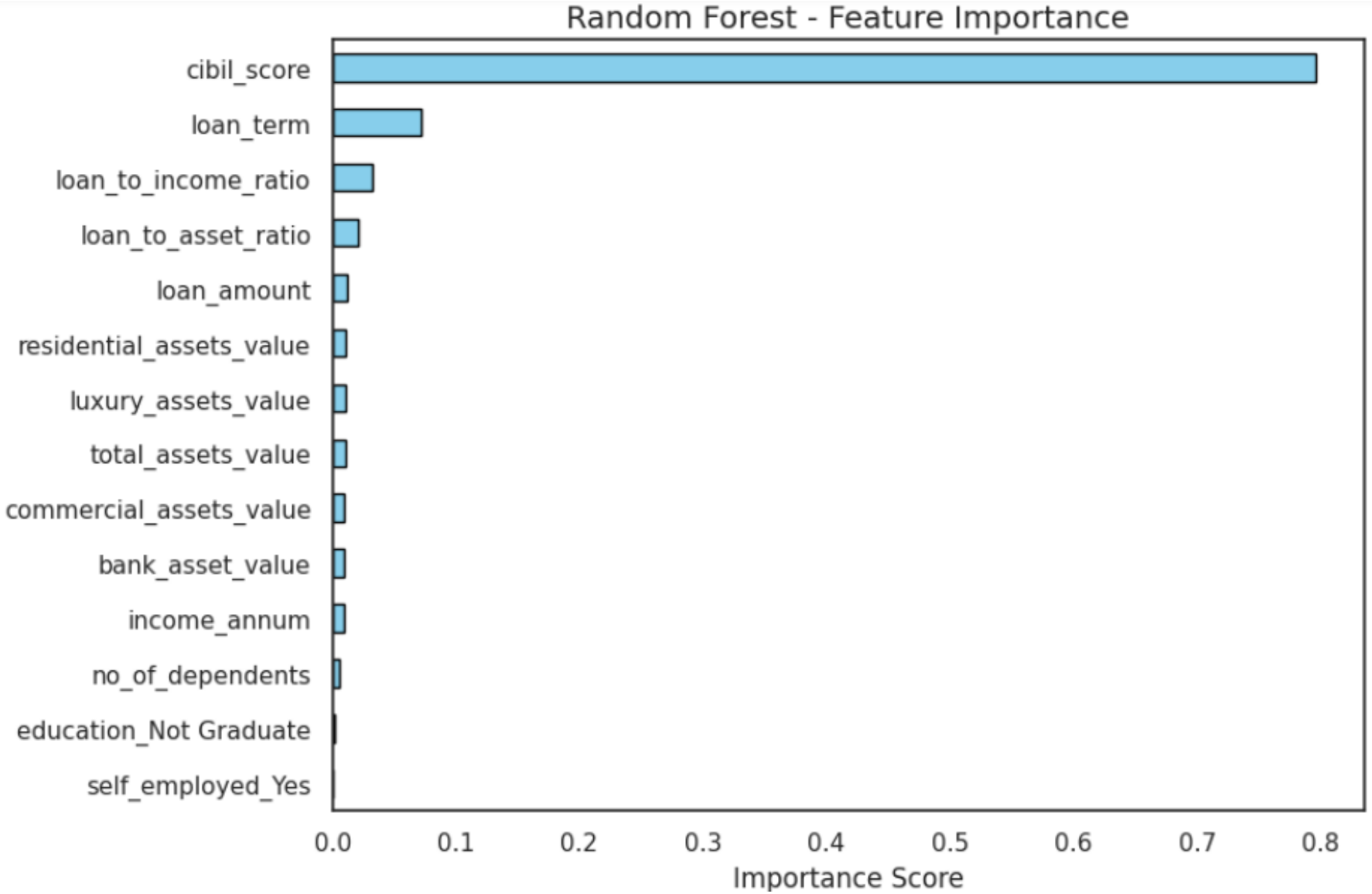
Model Benchmark Results Analysis

Logistic Regression (Baseline): Achieved **90.75% accuracy** and **0.9727 ROC-AUC**, proving to be a highly reliable and interpretable linear baseline.

Tree-Based Models (RF & XGBoost): Both models achieved near-perfect performance (~**99.8% accuracy** and **1.000 ROC-AUC**), demonstrating exceptional capacity to capture complex decision boundaries.

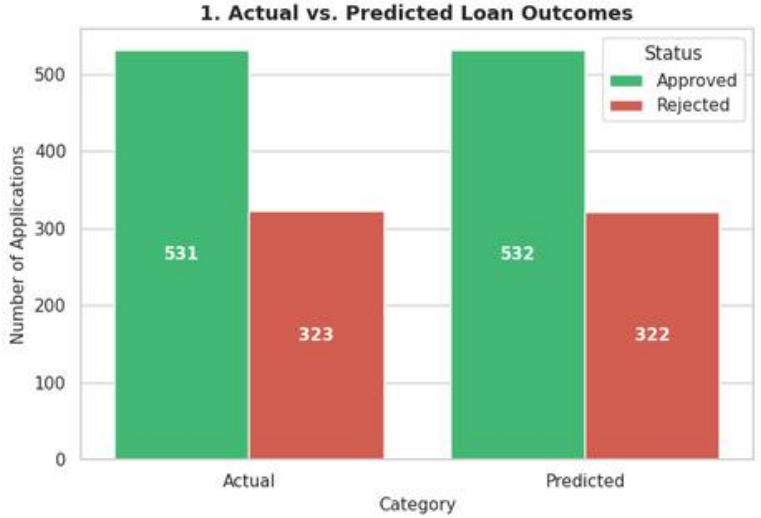
Balanced Performance: High precision ($0.92+$) and recall ($0.94+$) across all models show strong performance in predicting both approved and rejected loans.

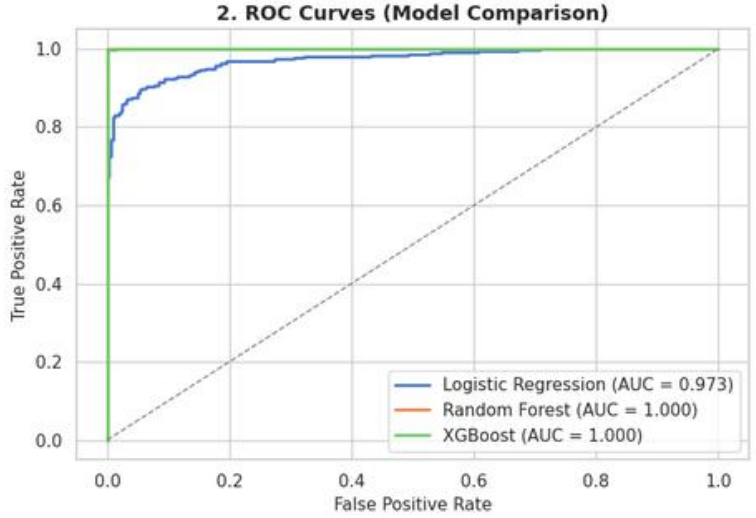
Key Report Note (Overfitting Caveat): The perfect AUC (1.0000) for tree models suggests potential overfitting to synthetic dataset rules, making **Logistic Regression** the most realistic model for production deployment without further regularization.

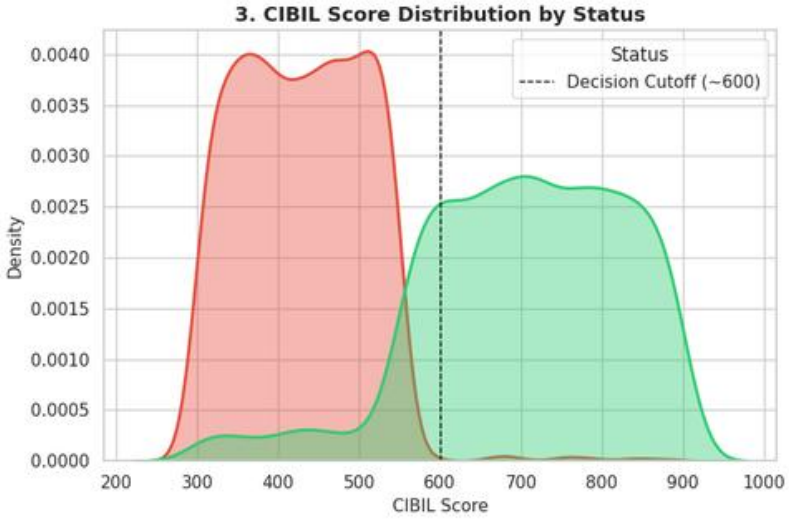


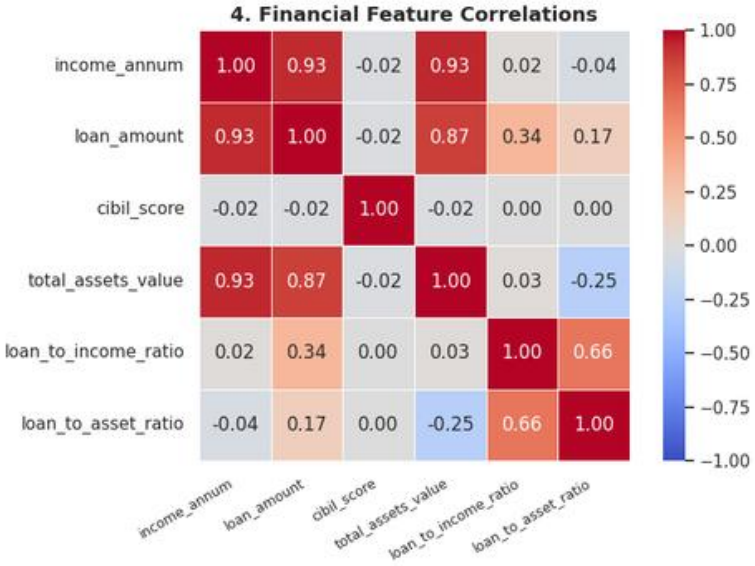
Creditworthiness (CIBIL score) dominates loan approval decisions with nearly 80% feature weight, while demographic factors like education and employment have virtually zero impact.

Core Model Analytics & Key Risk Factors









Across all four analytics panels, the models demonstrate near-perfect predictive capability; achieving up to a 1.000 ROC-AUC score and matching actual loan outcomes almost identically with 532 predicted approvals and 322 rejections. The primary driver behind this performance is a clear CIBIL score threshold at approximately 600, where applicants below this mark face high rejection rates and those above are consistently approved. Furthermore, while core financial features like income, loan amount, and asset values display strong internal correlation ($r \geq 0.87$), creditworthiness (CIBIL score) operates completely independently ($r \approx 0.00$) as the ultimate deciding factor.

Conclusion & Strategic Takeaways

Key Insights & Operational Impact

- **Credit Score Dominates Risk Assessment:** Feature importance analysis reveals that **`cibil_score` drives ~80% of the decision**, enforcing a sharp cutoff at **~600**.

- **Zero Demographic Bias:** Non-financial factors (education, employment status, dependents) hold **~0% importance**, ensuring completely objective and non-discriminatory evaluations.

- **Model Selection for Production:**
  - While tree models (Random Forest/XGBoost) hit ~100% training scores, **Logistic Regression (90.75% accuracy | 0.9727 ROC-AUC)** provides the most dependable, interpretable, and non-overfitted choice for live deployment.

- **Business Impact:**
  - **Instant Turnaround:** Cuts application processing time from days to milliseconds.
  - **Fairness & Consistency:** Removes human bias while maintaining high classification accuracy on high-risk applicants.